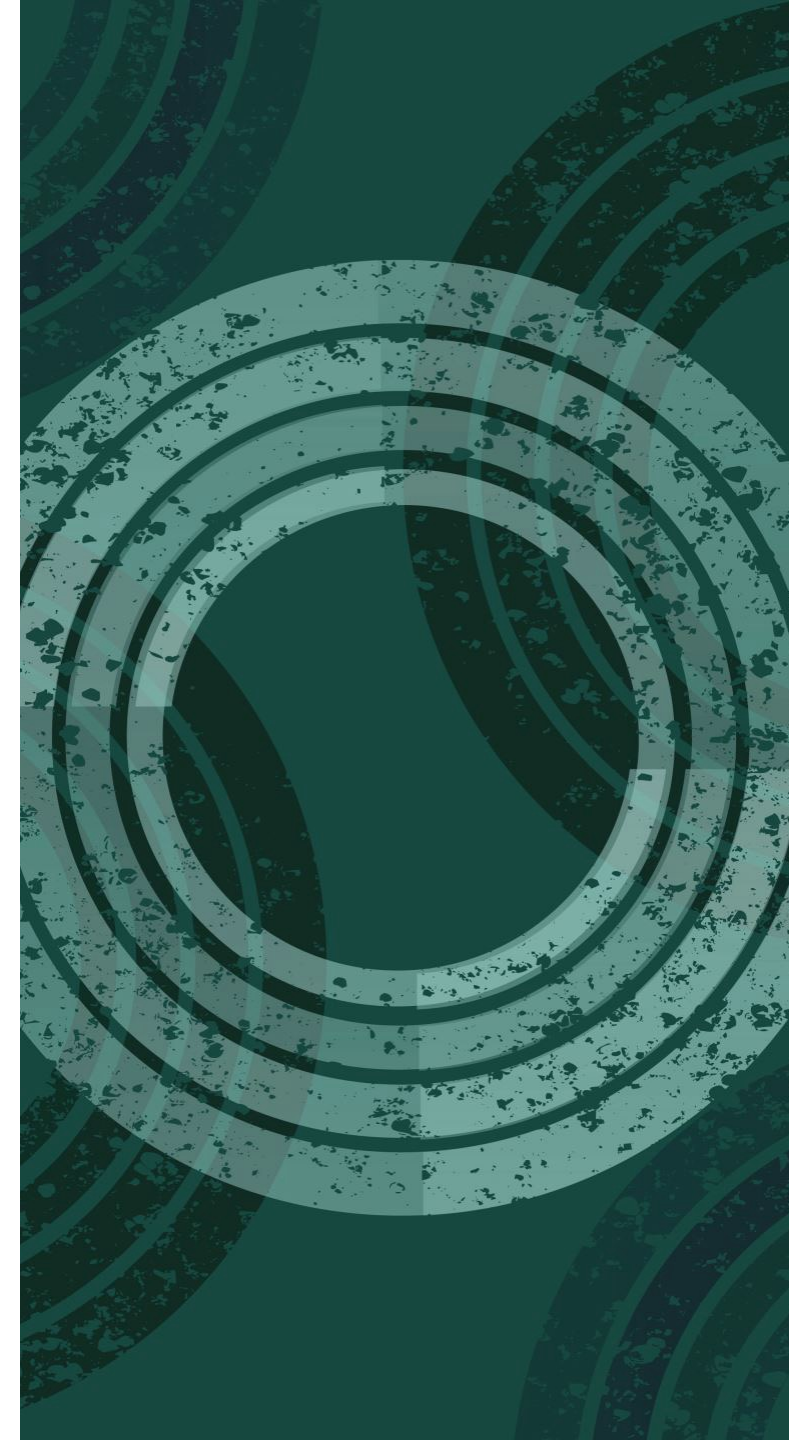
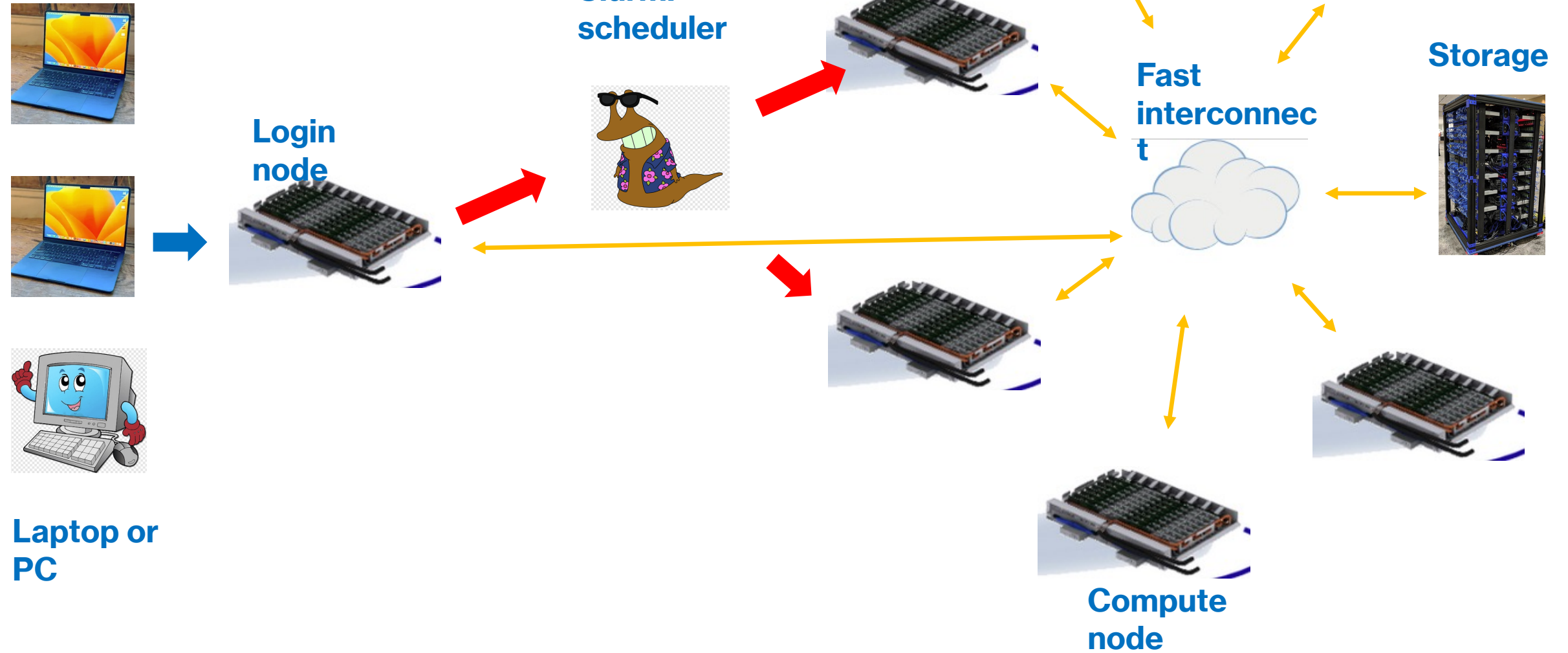


— INTRODUCTION TO GPU PARTITIONS ON ALPINE

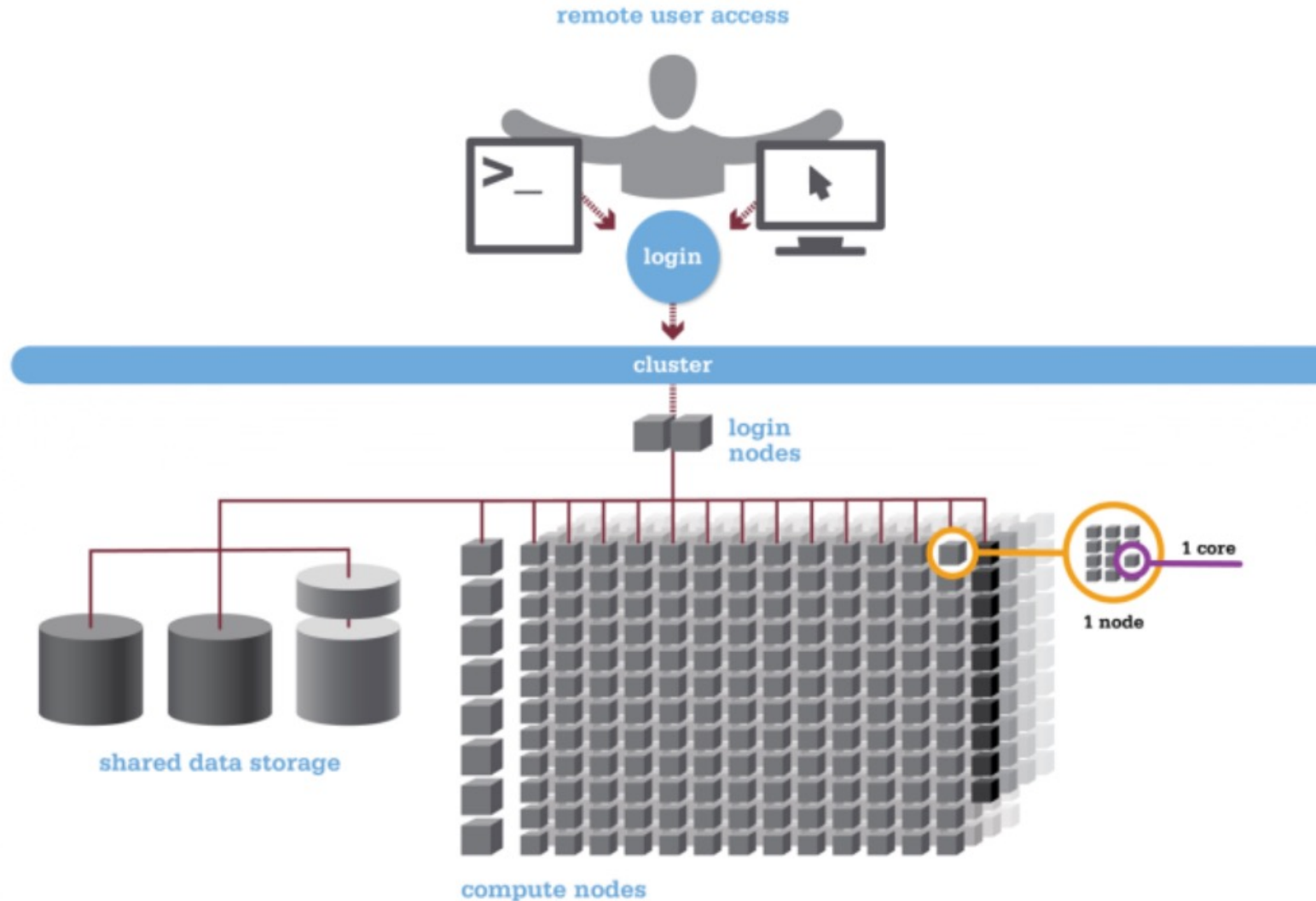
By Kevin Fotso



Anatomy of supercomputing



Architecture of a supercomputer

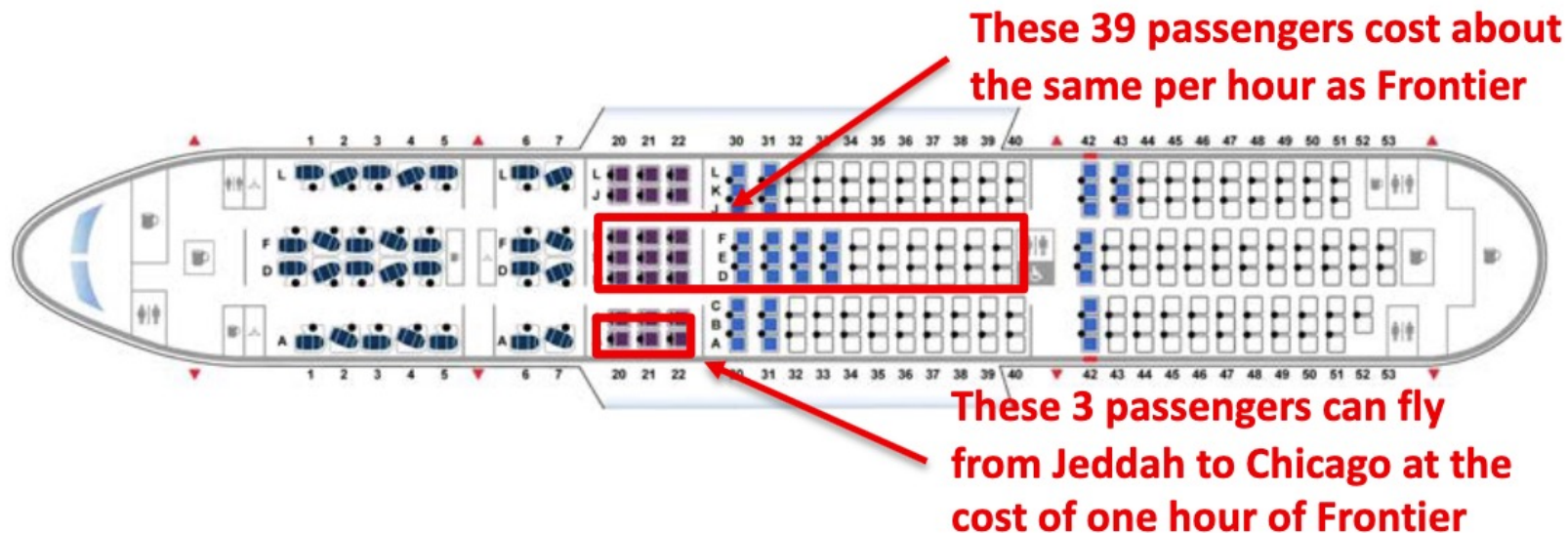


- Login nodes. To log into the system, cd into directories, look at files etc ...
- Compute node. Dedicated to do the computation
- The slurm scheduler controls access to the compute nodes to avoid a tragedy of the commons ...

Why Slurm?

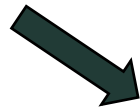


- Allows for better tracking of resource used per user
- Better management of of resource across user base
- More sustainable and less tragedy of the commons
- Widely used and supported by applications in case of bugs.



Why Slurm?

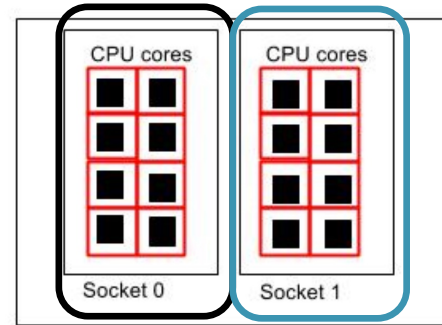
- Bulma request 8 cores from Node 1
- Dr. Brief requests 8 cores as well from Node 1
- Gohan requests 16 cores from Node 2
- No tragedy of the common!!!



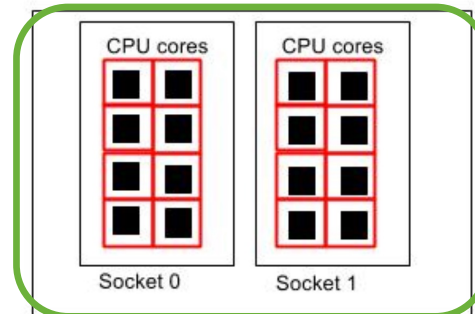
**Slurm
scheduler**



Node 1



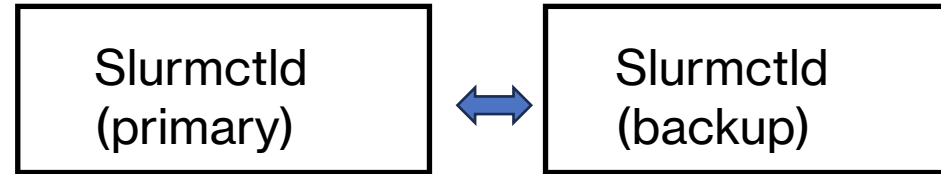
Node 2



Slurm architecture

- Slurmctld are daemons that monitor the state of resources, decide when to initiate job states and process users command.

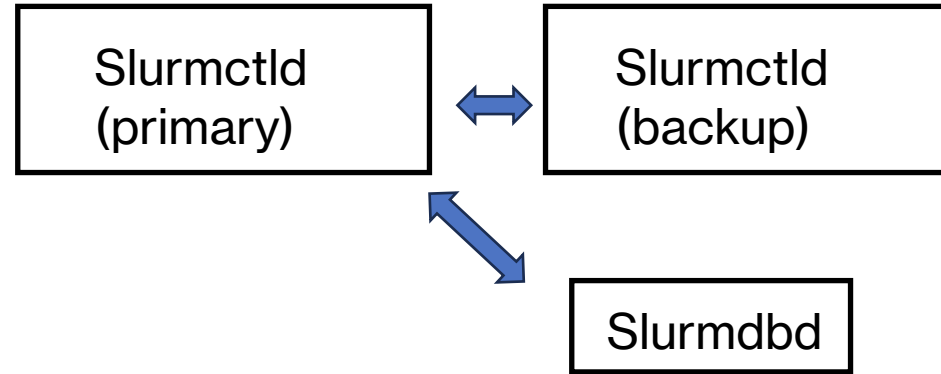
Controler daemons



Slurm architecture

- Slurmctld are daemons that monitor the state of resources, decide when to initiate job states and process users command.
- Slurmdbd is the database daemon and records accounting information and manages some other config (qos, fair share ...)

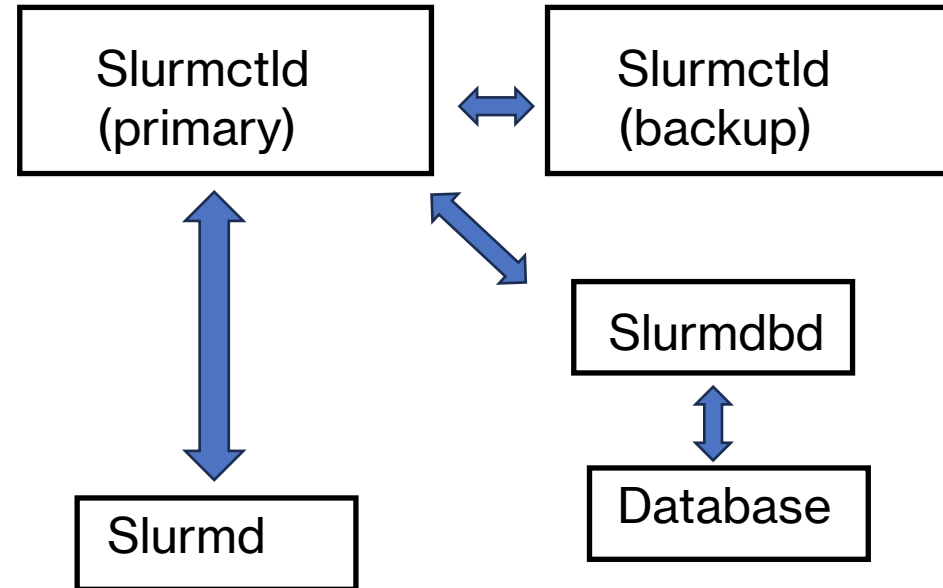
Controler daemons



Slurm architecture

- Slurmctld are daemons that monitor the state of resources, decide when to initiate job states and process users command.
- Slurmdbd is the database daemon and records accounting information and manages some other config (qos, fair share ...).
- Slurmd is the slurm compute daemon on compute nodes. Spawn processes on behalf of users

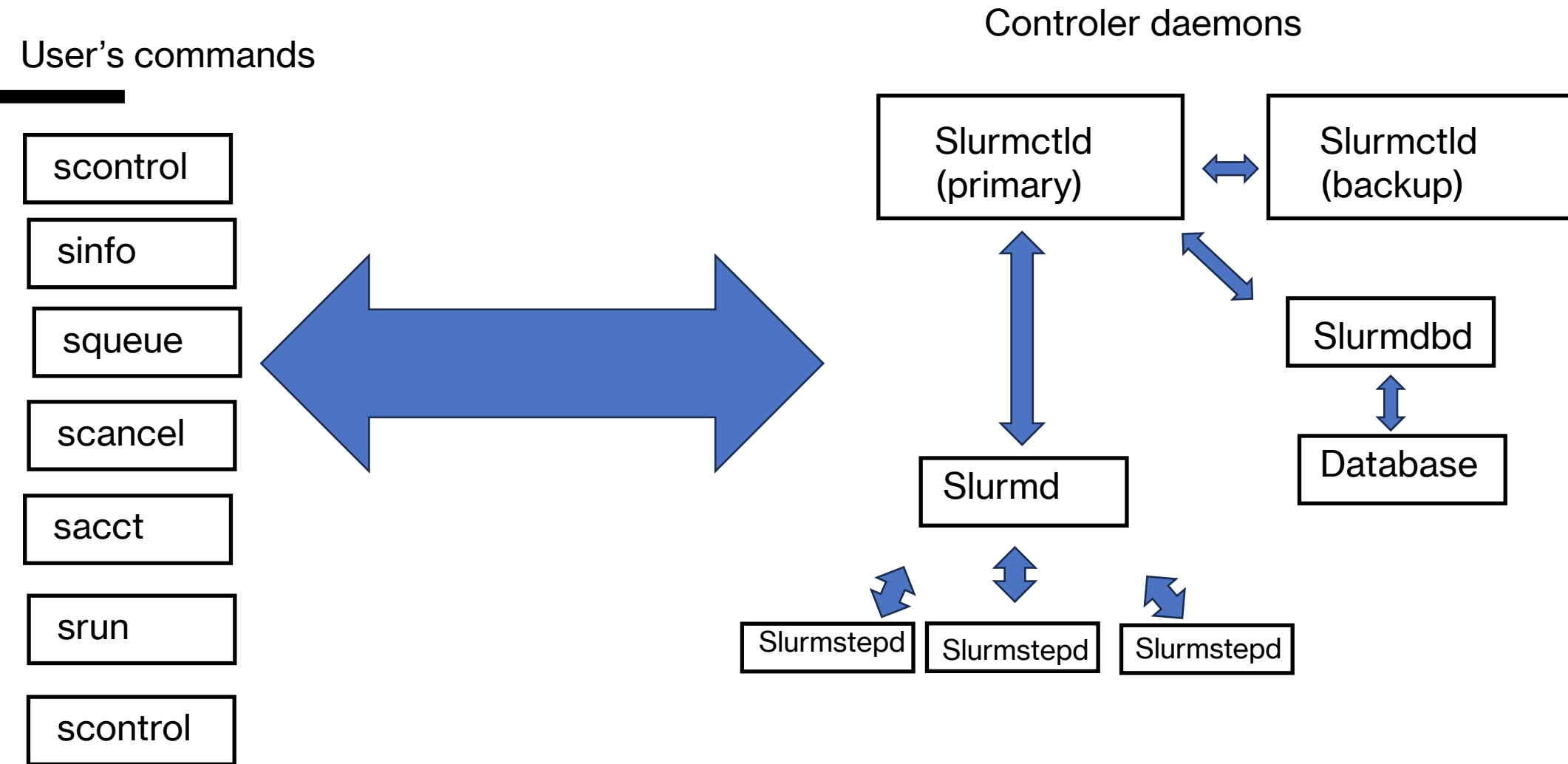
Controler daemons



- ████████████████████



Slurm architecture



Context [1]

- Our cluster named Alpine is made of 32,368 cores and 489 compute nodes

Context [1]

- Our cluster named Alpine is made of 32,368 cores and 489 compute nodes
- We have 12 A100, 3 L40, 2 Grace Hopper, 8 H200 and 8 RTXpro6000 NVIDIA gpu nodes as well as 8 AMD MI100 AMD gpu nodes.
- Each GPU node is 2x25 Gb Ethernet+RoCE interconnected.

Context [1]

- Our cluster named Alpine is made of 32,368 cores and 489 compute nodes
- We have 12 A100, 3 L40, 2 Grace Hopper, 8 H200 and 8 RTXpro6000 NVIDIA gpu nodes as well as 8 AMD MI100 AMD gpu nodes.
- Each GPU node is 2x25 Gb Ethernet+RoCE interconnected.
- 3 gpus per node, except for Grace Hopper (1 gpu per node), the H200 (4 gpus per node) and RTXpro6000 (4 gpus per node).

Context [1]

- Our cluster named Alpine is made of 32,368 cores and 489 compute nodes
- We have 12 A100, 3 L40, 2 Grace Hopper, 8 H200 and 8 RTXpro6000 NVIDIA gpu nodes as well as 8 AMD MI100 AMD gpu nodes.
- Each GPU node is 2x25 Gb Ethernet+RoCE interconnected.
- 3 gpus per node, except for Grace Hopper (1 gpu per node), the H200 (4 gpus per node) and RTXpro6000 (4 gpus per node).
- The OS is RedHat 8.10
- One needs to submit a proposal to access Grace Hopper. Let me know if interested!

Software stack

- Our software versions on the cuda side are: [cuda/11.2](#), [cuda/11.3](#), [cuda/11.4](#), [cuda/11.8](#) and [cuda/12.1.1](#).

Software stack

- Our software versions on the cuda side are: [cuda/11.2](#), [cuda/11.3](#), [cuda/11.4](#), [cuda/11.8](#) and [cuda/12.1.1](#)
- Our software versions on the AMD side are: [rocm/5.2.3](#), [rocm/5.3.0](#), [rocm/5.5.0](#), [rocm/5.6.0](#), [rocm/6.1.0](#)

QOS changes on Alpine

- To improve resource availability across Alpine GPU partitions and simplify job submissions, several QOS changes have been made.

QOS changes on Alpine

Quality of service (QOS) type	Description
normal	For a job with a wall time of up to 24 hours and in batch mode.
long	A job with a walltime between 24 hours and 7 days and in batch mode.
testing	For jobs that interact with a gpu interactively and not in batch mode



QOS types before the July maintenance.

QOS changes on Alpine

- The new QOS combines the old qos definition with the generic resource (GRES) type in order to reduce very long wait times.

QOS changes on Alpine

Quality of service (QOS) type	Description
gpu-normal	For a job with a wall time of up to 24 hours and in batch mode.
gpu-long	A job with a walltime between 24 hours and 7 days and in batch mode.
gpu-testing	For jobs that interact with a gpu interactively and not in batch mode



QOS types **after** the July maintenance.

Recap

Quality of service (QOS) type	Description
normal	For a job with a wall time of up to 24 hours and in batch mode.
long	A job with a walltime between 24 hours and 7 days and in batch mode.
testing	For jobs that interact with a gpu interactively and not in batch mode



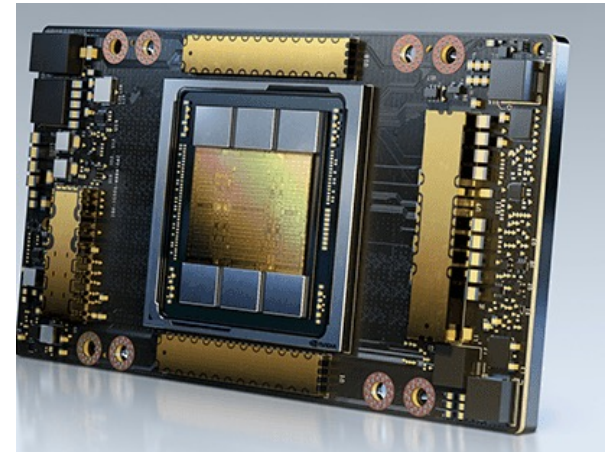
Quality of service (QOS) type	Description
gpu-normal	For a job with a wall time of up to 24 hours and in batch mode.
gpu-long	A job with a walltime between 24 hours and 7 days and in batch mode.
gpu-testing	For jobs that interact with a gpu interactively and not in batch mode

Computing situation

- Our users on the Medical Campus side have been used to computation on the A100 nodes, and more recently L40 nodes.

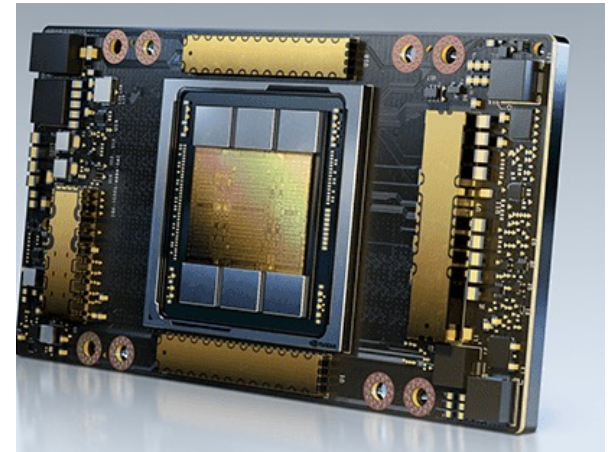
Alpine NVIDIA A100 Specs

- 40GB or 80 GB global memory for each gpu.



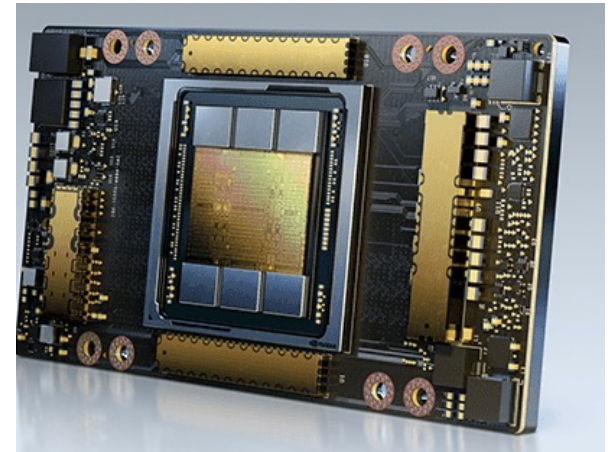
Alpine NVIDIA A100 Specs

- 40GB or 80 GB global memory for each gpu.
- Peak memory bandwidth is >2TB/s.



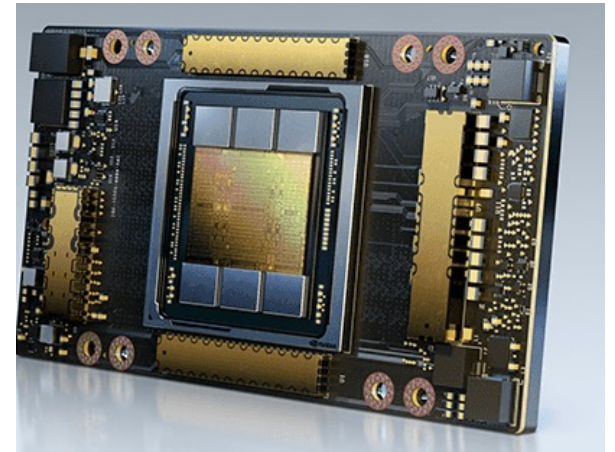
Alpine NVIDIA A100 Specs

- 40GB or 80 GB global memory for each gpu.
- Peak memory bandwidth is >2TB/s.
- 3 NVIDIA gpus per node



Alpine NVIDIA A100 Specs

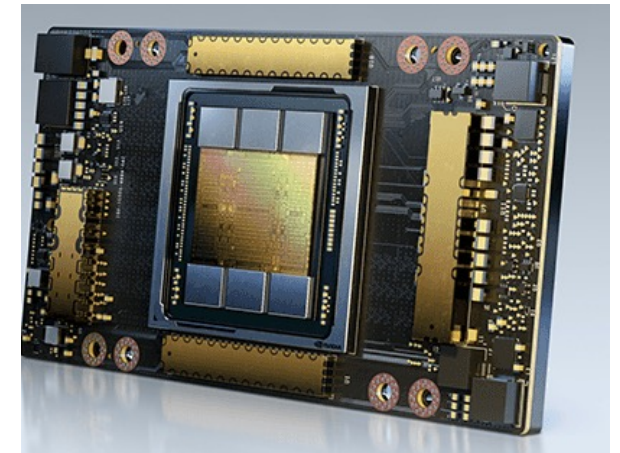
- 40GB or 80 GB global memory for each gpu.
- Peak memory bandwidth is >2TB/s.
- 3 NVIDIA gpus per node
- One partition name on Alpine: aa100 for both batch submission and interactive debugging.



Alpine NVIDIA A100 Specs

To access a full A100 gpu with 40GB please include the correct partition and gres types as follows:

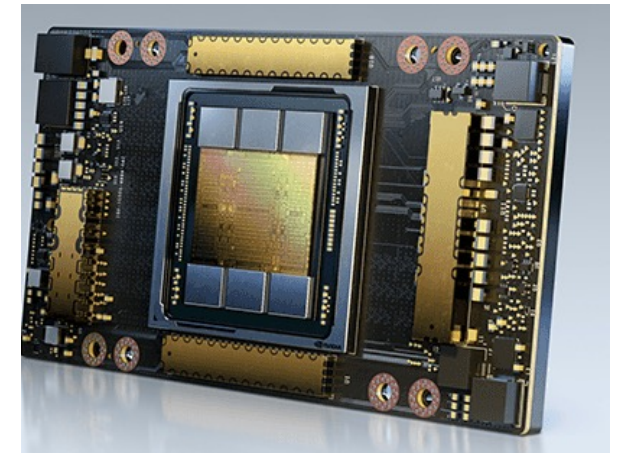
- **#SBATCH --partition=aa100**
#SBATCH --gres=gpu:a100-40gb:1
- **#SBATCH --qos=gpu-normal**



Alpine NVIDIA A100 Specs

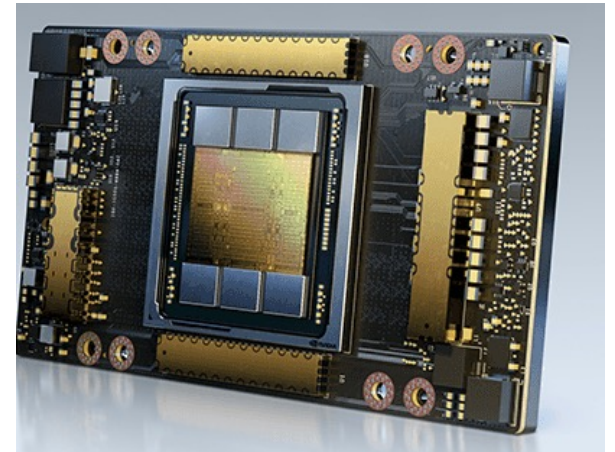
Similarly, to access a full A100 gpu with 80GB please include the correct partition and gres types as follows:

- **#SBATCH --partition=aa100**
#SBATCH --gres=gpu:a100_80gb:1
- **#SBATCH --qos=gpu-normal**



Alpine NVIDIA L40 Specs

- 48GB global memory for each gpu.
- Peak memory bandwidth is 864 GB/s.
- 3 NVIDIA gpus per node
- One partition names on Alpine: al40 for batch submission



Alpine NVIDIA L40 Specs

To access a full L40 gpu with 48G please include the correct partition and gres types as follows:

- **#SBATCH --partition=al40**
#SBATCH --gres=gpu:l40:1
#SBATCH --qos=gpu-normal

Alpine NVIDIA H200 Specs

- 141GB global memory for each gpu.
- Peak memory bandwidth is 4.8 TB/s.
- 4 NVIDIA gpus per node
- One partition name on Alpine: ah200 for batch submission

Alpine NVIDIA H200 Specs

To access **a full** h200 gpu with 141GB please include the correct partition and gres types as follows:

- **#SBATCH --partition=ah200**
#SBATCH --gres=gpu:h200:1
#SBATCH --qos=gpu-normal

Alpine NVIDIA RTX pro 6000 specs

- 96 GB global memory for each gpu.
- Peak memory bandwidth is 1.792 TB/s.
- 4 NVIDIA gpus per node
- One partition name on Alpine: artxpro6000 for batch submission

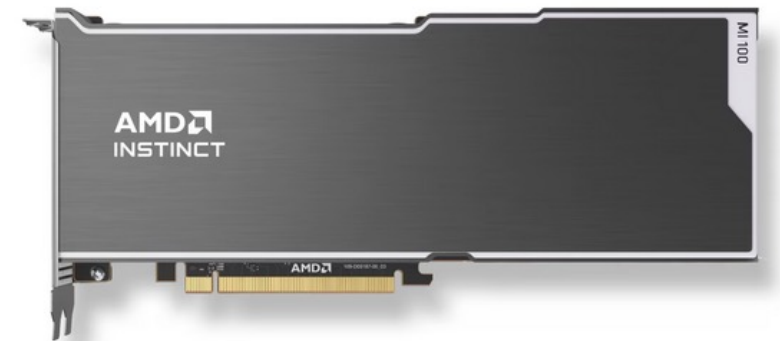
Alpine NVIDIA RTX pro 6000 specs

To access a **full rtx6000 pro** gpu with 96GB please include the correct partition and gres types as follows:

- **#SBATCH --partition=artxpro6000**
#SBATCH --gres=gpu:rtx_pro_6000:1
#SBATCH --qos=gpu-normal

Alpine AMD MI100 Specs

- 32 GB global memory for each gpu (up to 80 GB memory for the NVIDIA A100 gpu)



Alpine AMD MI100 Specs

- 32 GB global memory for each gpu (up to 80 GB memory for the NVIDIA A100 gpu)
- Peak memory bandwidth is 1.2 TB/s (>2TB/s for NVIDIA A100)



Alpine AMD MI100 Specs

- 32 GB global memory for each gpu (up to 80 GB memory for the NVIDIA A100 gpu)
- Peak memory bandwidth is 1.2 TB/s (>2TB/s for NVIDIA A100)
- 3 AMD gpus per node



Alpine AMD MI100 Specs

- 32 GB global memory for each gpu (up to 80 GB memory for the NVIDIA A100 gpu)
- Peak memory bandwidth is 1.2 TB/s (>2TB/s for NVIDIA A100)
- 3 AMD gpus per node
- One partition names on Alpine: ami100 for batch submission.



Alpine AMD MI100 Specs

- To access the mi100 gpu with 32G please include the correct partition and gres types as follows:

- **#SBATCH --partition=ami100**
#SBATCH --gres=gpu:mi100:1
- **#SBATCH --qos=gpu-normal**



Slurm & GPU partitions

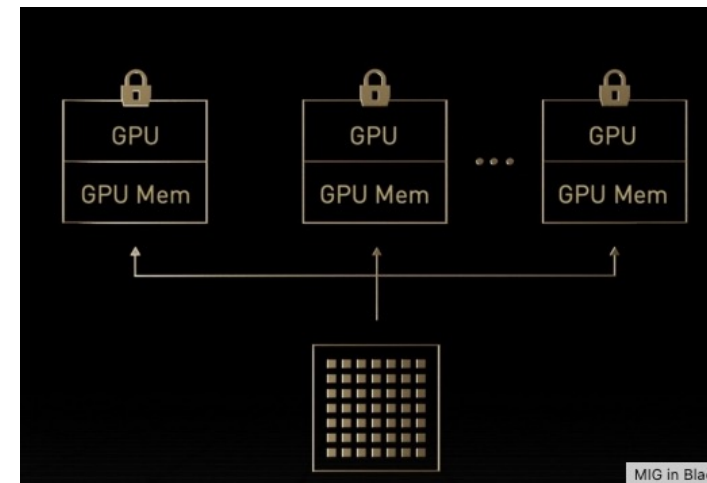
- `sinfo --Format partition`
- `sinfo --partition aa100,ami100 --Format=Partition,Nodes,Time`
- `scontrol show partition aa100`
- `scontrol show node c3gpu-c2-u13`



To print current state, type of nodes, default walltime, partition name and information, total nodes etc ...

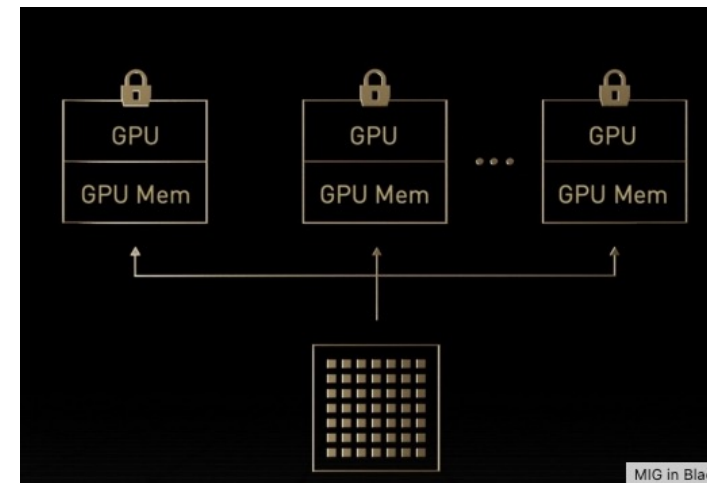
NVIDIA MIG

- The aa100 partition utilizes NVIDIA's [Multi-Instance GPU \(MIG\)](https://docs.nvidia.com/datacenter/tesla/mig-user-guide/latest/concepts.html) feature, which can “slice” GPUs into multiple GPU instances: <https://docs.nvidia.com/datacenter/tesla/mig-user-guide/latest/concepts.html>



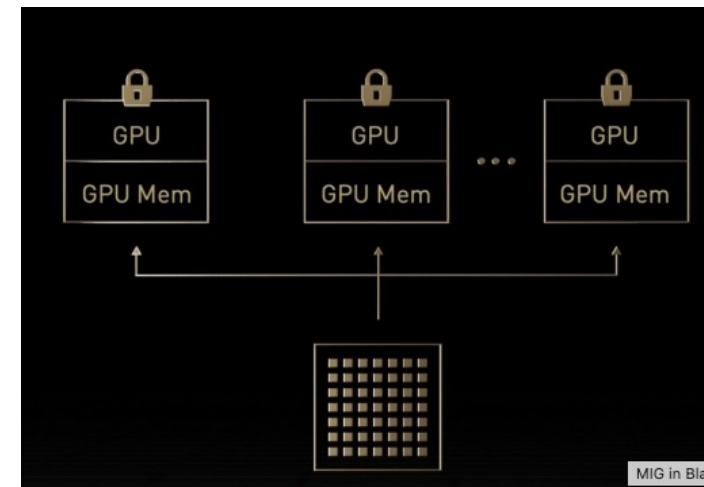
NVIDIA MIG

- The aa100 partition utilizes NVIDIA's [Multi-Instance GPU \(MIG\)](https://docs.nvidia.com/datacenter/tesla/mig-user-guide/latest/concepts.html) feature, which can “slice” GPUs into multiple GPU instances: <https://docs.nvidia.com/datacenter/tesla/mig-user-guide/latest/concepts.html>
- These GPU instances can be treated as a single GPU.



NVIDIA MIG

- The aa100 partition can now utilize the NVIDIA's [Multi-Instance GPU \(MIG\)](https://docs.nvidia.com/datacenter/tesla/mig-user-guide/latest/concepts.html) feature, which can “slice” GPUs into multiple GPU instances: <https://docs.nvidia.com/datacenter/tesla/mig-user-guide/latest/concepts.html>
- These GPU instances can be treated as a single GPU.
- One important limitation is that MIG does not allow for multiple GPU instances to communicate with each other.



NVIDIA MIG

- To access a gpu for debugging purposes, most gres call will adopt a structure similar to this:
- `--gres=gpu:<gpu_name>_compute_slice.memory_slice:total_gpus_requested`

NVIDIA MIG

- For example:

`--gres=gpu:a100_3g.20gb:1.`



Generic
resources

NVIDIA MIG

- For example:

`--gres=gpu:100_3g.20gb:1.`



Compute
slice

NVIDIA MIG

- For example:

`--gres=gpu:a100_3g.20gb:1.`



Memory
slice

NVIDIA MIG for an h200 gpu

To access only a 35gb slice of an h200 gpu please include the correct partition and gres types as follows:

```
#SBATCH --partition=ah200  
#SBATCH --gres=gpu:h200_2g.35gb:1  
#SBATCH --qos=gpu-normal
```

NVIDIA MIG for an h200 gpu

To access only a 71gb slice of an h200 gpu please include the correct partition and gres types as follows:

```
#SBATCH --partition=ah200  
#SBATCH --gres=gpu:h200_3g.71gb:1  
#SBATCH --qos=gpu-normal
```

NVIDIA MIG for an RTX PRO 6000 gpu

To access only a 24gb slice of an rtx pro 6000 gpu please include the correct partition and gres types as follows:

```
#SBATCH --partition=artxpro6000  
#SBATCH --gres=gpu:rtx_pro_6000_1g.24gb:1  
#SBATCH --qos=gpu-normal  
#SBATCH --ntasks=8
```



You can only request a maximum of 8 CPU tasks on this partition!

NVIDIA MIG for an RTX PRO 6000 gpu

To access only a 48gb slice of an rtx pro 6000 gpu please include the correct partition and gres types as follows:

```
#SBATCH --partition=artxpro6000  
#SBATCH --gres=gpu:rtx_pro_6000_2g.48gb:1  
#SBATCH --qos=gpu-normal  
#SBATCH --ntasks=8
```



You can only request a maximum of 8 CPU tasks on this partition!

NVIDIA MIG

- Below are the full documented changes to access a gpu for debugging (interactive) purposes:

Before (up through July 2026)	After(August 2026 – present)
<code>sinteractive --partition=atesting_a100 --gres=gpu:1 --qos=testing --time=00:30:00 --nodes=1 --ntasks=4</code>	<code>sinteractive --partition=aa100 --gres=gpu:a100_3g.20gb:1 --qos=gpu-normal --time=00:30:00 --nodes=1 --ntasks=4</code>
<code>sinteractive --partition=atesting_ami100 --gres=gpu:1 --qos=testing --time=00:30:00 --nodes=1 --ntasks=4</code>	<code>sinteractive --partition=ami100 --gres=gpu:mi100:1 --qos=gpu-normal --time=00:30:00 --nodes=1 --ntasks=4</code>

Full slurm scripts

Documented changes on how to have access to an a100 (40G) gpu in slurm batch mode.

Before: (up through July 2026)

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks=4
#SBATCH --nodes=1
#SBATCH --account=amc-general
#SBATCH --partition=aa100
#SBATCH --time=00:20:00
#SBATCH --output=FFT.%j.out
#SBATCH --error=FFT.%j.err
#SBATCH --qos=normal
#SBATCH --gres=gpu:1
```



Now: (August 2026 – present)

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks=4
#SBATCH --nodes=1
#SBATCH --account=amc-general
#SBATCH --partition=aa100
#SBATCH --time=00:20:00
#SBATCH --output=FFT.%j.out
#SBATCH --error=FFT.%j.err
#SBATCH --qos=gpu-normal
#SBATCH --gres=gpu:a100-40gb:1
```

Full slurm scripts

Documented changes on how to have access to an a100 (80G) gpu in slurm batch mode.

Before: (up through July 2026)

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks=4
#SBATCH --nodes=1
#SBATCH --account=amc-general
#SBATCH --partition=aa100
#SBATCH --time=00:20:00
#SBATCH --output=FFT.%j.out
#SBATCH --error=FFT.%j.err
#SBATCH --qos=normal
#SBATCH --gres=gpu:1
```



Now: (August 2026 – present)

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks=4
#SBATCH --nodes=1
#SBATCH --account=amc-general
#SBATCH --partition=aa100
#SBATCH --time=00:20:00
#SBATCH --output=FFT.%j.out
#SBATCH --error=FFT.%j.err
#SBATCH --qos=gpu-normal
#SBATCH --gres=gpu:a100_80gb:1
```

Full slurm scripts

Documented changes on how to have access to an L40 (48G) gpu in slurm batch mode.

Before: (up through July 2026)

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks=4
#SBATCH --nodes=1
#SBATCH --account=amc-general
#SBATCH --partition=al40
#SBATCH --time=00:20:00
#SBATCH --output=FFT.%j.out
#SBATCH --error=FFT.%j.err
#SBATCH --qos=normal
#SBATCH --gres=gpu:1
```



Now: (August 2026 – present)

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks=4
#SBATCH --nodes=1
#SBATCH --account=amc-general
#SBATCH --partition=al40
#SBATCH --time=00:20:00
#SBATCH --output=FFT.%j.out
#SBATCH --error=FFT.%j.err
#SBATCH --qos=gpu-normal
#SBATCH --gres=gpu:l40:1
```

Full slurm scripts

Documented changes on how to have access to an MI100 (32G) gpu in slurm batch mode.

Before: (up through July 2026)

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks=4
#SBATCH --nodes=1
#SBATCH --account=amc-general
#SBATCH --partition=ami100
#SBATCH --time=00:20:00
#SBATCH --output=FFT.%j.out
#SBATCH --error=FFT.%j.err
#SBATCH --qos=normal
#SBATCH --gres=gpu:1
```



Now: (August 2026 – present)

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks=4
#SBATCH --nodes=1
#SBATCH --account=amc-general
#SBATCH --partition=ami100
#SBATCH --time=00:20:00
#SBATCH --output=FFT.%j.out
#SBATCH --error=FFT.%j.err
#SBATCH --qos=gpu-normal
#SBATCH --gres=gpu:mi100:1
```

Best practices

What happens if my gpu deep learning job runs out of memory?

- Assess your network size based on the number of parameters and the batch size, then multiply it by the precision type.

What happens if my gpu deep learning job runs out of memory?

- Assess your network size based on the number of parameters and the batch size, then multiply it by the precision type.
- For example, if your network is using **271,210** parameters with **float64** precision and **1** batch size, the calculation goes as follows:

[Mem_size= **271,210** parameters **X 8** bytes **X 1** batch size]

What happens if my gpu deep learning job runs out of memory?

- Assess your network size based on the number of parameters and the batch size, then multiply it by the precision type.
- For example, if your network is using **271,210** parameters with **float64** precision and **1** batch size, the calculation goes as follows:

[Mem_size= **271,210** parameters **X 8** bytes **X 1** batch size]

What happens if my gpu deep learning job runs out of memory?

- Consider as well reducing the precision, for instance from float64 to float32

What happens if my gpu deep learning job runs out of memory?

- Consider as well reducing the precision, for instance from float64 to float32.
- Reduce batch size.

What happens if my gpu deep learning job runs out of memory?

- Consider as well reducing the precision, for instance from float64 to float32.
- Reduce batch size.
- Consider doing distributed computing.

What happens if my gpu deep learning job runs out of memory?

- Consider as well reducing the precision, for instance from float64 to float32.
- Reduce batch size.
- Consider doing distributed computing.
- Consider doing feature selection prior to running your model in order to reduce the size of your data.

What happens if my gpu deep learning job runs out of memory?

- If you are using data loaders with Pytorch, consider increasing the number of workers:

`(#SBATCH --cpus-per-task=10 #SBATCH --ntasks=1).`

- If you are using Pytorch, consider also adjusting the garbage collection and expandable segments parameters as follows:

```
os.environ["PYTORCH_CUDA_ALLOC_CONF"] = (  
    "expandable_segments:True,"  
    "garbage_collection_threshold:0.9"  
)
```

Monitoring tools (NVIDIA gpus)

- Nvtop (pip install nvtop)
- gpustats (pip install gpustats)
- nvidia-smi

NVITOP 1.4.2				Driver Version: 570.124.06		CUDA Driver Version: 12.8	
GPU Fan	Name Temp Perf	Persistence-M Pwr:Usage/Cap	Bus-Id	Disp.A Memory-Usage	MIG M. GPU-Util	Uncorr. ECC Compute M.	
0 N/A	A100-PCIE-40GB 29C P0	On 33W / 250W	00000000:21:00.0	Off 212.9MiB / 40.00GiB	Enabled N/A	0 Default	MEM: 0.5% UTL: N/A
0:0	3g.20gb @ GI/CI: 2/0		106.4MiB / 20096MiB		BAR1:	64KiB / 0%	MEM: 0.5%
1 N/A	A100-PCIE-40GB 28C P0	On 34W / 250W	00000000:81:00.0	Off 212.9MiB / 40.00GiB	Enabled N/A	0 Default	MEM: 0.5% UTL: N/A
2 N/A	A100-PCIE-40GB 29C P0	On 35W / 250W	00000000:E2:00.0	Off 212.9MiB / 40.00GiB	Enabled N/A	0 Default	MEM: 0.5% UTL: N/A
Load Average: 0.88 0.23 0.08 CPU: 0.4%							AVG GPU MEM: 0.5%

kfotso@xsede.org@c3gpu-c2-u13.rc.int.colorado.edu

Run slurm script (NVIDIA GPU)

- sbatch **horovod_mpi.sh**
- Check the nodes where the job is running

```
user@login-ci2 ~$ squeue -u kfotso
JOBID PARTITION NAME USER ST TIME NODES NODELIST(REASON)
81362 aa100 horovod_ kfotso@x R 2:34 2 c3gpu-a9-u31-1,c3gpu-a9-u31-2
79946 acompile acompile kfotso@x R 2:03:48 1 c3cpu-c9-u5-2
81778 atesting_ sinterac kfotso@x R 7:56 1 c3gpu-a9-u29-1
user@login-ci2 ~$
```

```
user@login-ci2 ~$ squeue --me
JOBID PARTITION NAME USER ST TIME NODES NODELIST(REASON)
81362 aa100 horovod_ kfotso@x R 2:34 2 c3gpu-a9-u31-1,c3gpu-a9-u31-2
79946 acompile acompile kfotso@x R 2:03:48 1 c3cpu-c9-u5-2
81778 atesting_ sinterac kfotso@x R 7:56 1 c3gpu-a9-u29-1
user@login-ci2 ~$ ssh c3gpu-a9-u29-1^C
```



We have a GPU
node that we can
ssh to:
c3gpu-a9-u29-1

GPU monitoring (NVIDIA GPU)

- ssh to a that node on a different screen and run:

`nvidia-smi -l`

NVIDIA-SMI 525.85.12 Driver Version: 525.85.12 CUDA Version: 12.0									
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC		
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute	M.		
						MIG	M.		
0	NVIDIA A100 80G...	On	00000000:25:00.0	Off	26%	0	Disabled		
N/A	33C	P0	80W / 300W	1869MiB / 81920MiB					
1	NVIDIA A100 80G...	On	00000000:81:00.0	Off	25%	0	Default		
N/A	32C	P0	81W / 300W	1869MiB / 81920MiB			Disabled		

Processes:								
GPU	GI	CI	PID	Type	Process name	GPU Memory Usage		
	ID	ID						
0	N/A	N/A	569301	C	python	1866MiB		
1	N/A	N/A	569302	C	python	1866MiB		

We can confirm
that we are using
2 GPUs!!!

GPU monitoring (NVIDIA GPU)

- ssh to a that node on a different screen and run:

```
nvidia-smi -l
```

NVIDIA-SMI 525.85.12 Driver Version: 525.85.12 CUDA Version: 12.0									
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC		
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute	M.		
						MIG	M.		
0	NVIDIA A100	80G...	On	00000000:25:00.0	Off	0			
N/A	33C	P0	80W / 300W	1869MiB / 81920MiB	26%	Disabled			
1	NVIDIA A100	80G...	On	00000000:81:00.0	Off	0			
N/A	32C	P0	81W / 300W	1869MiB / 81920MiB	25%	Default			
						Disabled			

Percentage GPU usability. That is the portion of time 1 or more kernels were used

Processes:							
GPU	GI	CI	PID	Type	Process name	GPU Memory Usage	
ID	ID	ID					
0	N/A	N/A	569301	C	python	1866MiB	
1	N/A	N/A	569302	C	python	1866MiB	

- **■ ■ ■**

+---

 $+ - - -$

How much
memory is used.

GPU monitoring (NVIDIA GPU)

- nvidia-smi has more parameters that can be used. Cheatsheet here:

<https://www.docs.arc.vt.edu/usage/gpumon.html>

```
[ xseide.org@c3gpu-a9-u29-1 ~]$ nvidia-smi --query-gpu=timestamp,name,pci.bus_id,driver_version,temperature.gpu,utilization.gpu,utilization.memory --format=csv -l 5
timestamp, name, pci.bus_id, driver_version, temperature.gpu, utilization.gpu [%], utilization.memory [%]
2023/10/23 21:42:45.228, NVIDIA A100 80GB PCIe, 00000000:25:00.0, 525.85.12, 32, 0 %, 0 %
2023/10/23 21:42:45.230, NVIDIA A100 80GB PCIe, 00000000:81:00.0, 525.85.12, 31, 0 %, 0 %
2023/10/23 21:42:50.233, NVIDIA A100 80GB PCIe, 00000000:25:00.0, 525.85.12, 33, 7 %, 0 %
2023/10/23 21:42:50.234, NVIDIA A100 80GB PCIe, 00000000:81:00.0, 525.85.12, 32, 5 %, 0 %
2023/10/23 21:42:55.236, NVIDIA A100 80GB PCIe, 00000000:25:00.0, 525.85.12, 34, 25 %, 2 %
2023/10/23 21:42:55.237, NVIDIA A100 80GB PCIe, 00000000:81:00.0, 525.85.12, 32, 23 %, 1 %
2023/10/23 21:43:00.238, NVIDIA A100 80GB PCIe, 00000000:25:00.0, 525.85.12, 34, 25 %, 2 %
2023/10/23 21:43:00.240, NVIDIA A100 80GB PCIe, 00000000:81:00.0, 525.85.12, 33, 25 %, 1 %
2023/10/23 21:43:05.241, NVIDIA A100 80GB PCIe, 00000000:25:00.0, 525.85.12, 34, 25 %, 2 %
2023/10/23 21:43:05.243, NVIDIA A100 80GB PCIe, 00000000:81:00.0, 525.85.12, 33, 24 %, 1 %
2023/10/23 21:43:10.244, NVIDIA A100 80GB PCIe, 00000000:25:00.0, 525.85.12, 35, 26 %, 2 %
2023/10/23 21:43:10.246, NVIDIA A100 80GB PCIe, 00000000:81:00.0, 525.85.12, 33, 25 %, 2 %
2023/10/23 21:43:15.247, NVIDIA A100 80GB PCIe, 00000000:25:00.0, 525.85.12, 35, 26 %, 2 %
2023/10/23 21:43:15.248, NVIDIA A100 80GB PCIe, 00000000:81:00.0, 525.85.12, 33, 26 %, 2 %
```

GPU monitoring (AMD gpu)

```
@xsede.org@login-ci1 workshop_amd_gpu]$ squeue --me
  JOBID PARTITION    NAME    USER ST       TIME  NODES NODELIST(REASON)
   5187143   acompile  acompile kfotso@x  R      10:03      1 c3cpu-c11-u3-4
   5187117     ami100  sinterac kfotso@x  R      20:16      1 c3gpu-c2-u17
[ @xsede.org@login-ci1 workshop_amd_gpu]$ █
```



We look for a node
where the GPU job
is running

GPU monitoring (AMD gpu)

```
@xsede.org@c3gpu-c2-u17 ~]$ watch -n 1 rocm-smi --showmeminfo vram
```



This shows that we are
on the gpu node

GPU monitoring (AMD gpu)

```
@xsede.org@c3gpu-c2-u17 ~]$ watch -n 1 rocm-smi --showmeminfo vram
```



This rocm-smi is the
equivalent of nvidia-smi

GPU monitoring (AMD gpu)

```
every 1.0s: rocm-smi --showmeminfo vram
```

```
=====ROCM System Management Interface=====
GPU[1]      : vram Total Memory (B): 34342961152
GPU[1]      : vram Total Used Memory (B): 2590732288
GPU[2]      : vram Total Memory (B): 34342961152
GPU[2]      : vram Total Used Memory (B): 7012352
GPU[3]      : vram Total Memory (B): 34342961152
GPU[3]      : vram Total Used Memory (B): 7012352
=====
=====End of ROCm SMI Log =====
```



We are on GPU 1 and we can tell that the job is using the GPU memory

How to estimate my GPU core hours?

- We need to be mindful of the gpu resources that we request on Alpine.
- Our campus has a total of 6,459 total shares which means that we can spend about 6,459 cores per hour in total before our priority starts to get affected.
- Let's see a scenario that highlights the gpu core hours calculation.

Scenario

- User A requests 1 full a100 (40G VRAM) gpu with 8 cpu ntasks for 1 hour
- The formula goes as follows:

Total SU = [(total_cores * TRESBillingWeightsCPU) + (total_cpu_mem * TRESBillingWeightsMem) + (total_gpus * GRES/gpu)] * total_hours

Scenario

- User A requests 1 full a100 (40G VRAM) gpu with 8 cpu ntasks for 1 hour
- The formula goes as follows:

Total SU = [(total_cores * TRESBillingWeightsCPU) + (total_cpu_mem * TRESBillingWeightsMem) + (total_gpus * GRES/gpu)] * total_hours

Total SU = [(8 * 0.5) + (28 GiB * 0.133) + (1 * 108.6)] * 1 hour

Scenario

- User A requests 1 full a100 (40G VRAM) gpu with 8 cpu ntasks for 1 hour
- The formula goes as follows:

Total SU = [(total_cores * TRESBillingWeightsCPU) + (total_cpu_mem * TRESBillingWeightsMem) + (total_gpus * GRES/gpu)] * total_hours

Total SU = [(8 * 0.5) + (28 GiB * 0.133) + (1 * 108.6)] * 1 hour

Total SU = 4 + 3.724 + 108.6

Total SU = 116.324 core hours.

Scenario

- You can find all gpu Billingweights here:

<https://curc.readthedocs.io/en/latest/clusters/alpine/alpine-hardware.html#available-gres-on-alpine>

- The calculation formula comes from this page:

<https://slurm.schedmd.com/tres.html>

Questions?